

SESSION - 15

Customizing AEM & Eclipse with AEM plugin Installation



Table of Contents

Table of Contents **1**

Custom Components ***2***

Customize the AEM UI ***2***

Overlays **2**

Sling Resource Merger **3**

Exercise 1: Extend a core component 3

Exercise 2: Extend the navigation UI 6

Exercise 3 : Hide a navigation button 7

Exercise 4: Add a new navigation button 7

Eclipse Installation **8**

AEM plugin Installation into Eclipse **8**

Installing AEM Developer Tools **10**

Exercise 5: Create an AEM project using latest archetype in eclipse 11

Custom Components

Introduction

During development on AEM projects there will be many such use cases where you have to customize AEM product and its UI. This session covers that aspect of AEM. This session is continuation of the previous session.

Objectives

After completing this module, you will be able to:

- Extend the AEM navigation UI
- Installing Eclipse with AEM plugin

Customize the AEM UI

The AEM UI is based on a set of nodes in JCR and its underlying structure. These nodes reside in the /libs folder. When you create a project, any customization required is done by overlaying the corresponding content node in the JCR under the /apps folder. This is done by overlaying the content node in the JCR.

The two methods used for customizations are:

Overlays

Sling Resource Merger

Overlays

Sling's Resource Resolver searches for resources in a list of specific paths, as defined by the Apache Sling JCR Resource Resolver. The default search path is first /apps and then /libs. As a result, you can change the out-of-the-box functionality as provided in /libs by adding a resource or file at the same path in /apps. This ability to override default functionality is called an overlay.

To use overlays in AEM:

1. Recreate the node structures from /libs under /apps.
2. Copy the content node that you want to change in /apps, and then work on this copy.
3. When a resource is requested, it is resolved according to the search path logic.
4. The Resource Resolver retrieves the resource from /apps, failing which, it looks at /libs.

An overlay involves taking the predefined functionality and imposing your own definitions over that to override the standard functionality.

Sling Resource Merger

The Sling Resource Merger is applicable only to the touch UI and enables you to customize consoles and page authoring. With Sling Resource Merger, you can overlay nodes without replicating all of their parent nodes.

The Sling Resource Merger provides services to access and merge resources. It merges the overlays of resources by using resource resolver search paths and difference mechanisms. A customized Sling vocabulary is used to manage the overrides of nodes and their properties. With this, the overlay resources/properties (defined in /apps) are merged with the original resources/properties (from /libs).

The Sling Resource Merger is used to ensure that all changes are made in /apps, and avoids the need to make any changes in /libs. After the structure is created, you can add your own properties to the nodes under /apps. The content of /apps has a higher priority than that of /libs. The properties defined in /apps indicate how the content merged from /libs are to be used.

The following table describes the differences between Overlays and Sling Resource Merger:

Overlays	Sling Resource Merger
Works based on search paths (/libs + /apps)	Works based on Resource Type Hierarchy
Must copy the whole subtree	Extends within an almost empty subtree
All properties are duplicated	Only required properties are overlaid
When upgrades are done to the /libs folder, changes need to be manually recreated under /apps.	As properties are not copied, only the structure, upgrades are automatically reflected in /apps

After you copy a system file from /libs to /apps to overlay it with custom code, your custom code will not pick up any modifications to the system component/file/script/dialog box, which results from the application of a hotfix, feature pack, or upgrade. A careful examination of the release notes of any upgrade, feature pack, or hotfix should be a step in your upgrade plan. This way, you will be able to evaluate changes and incorporate them into your application.

Exercise 1: Extend a core component

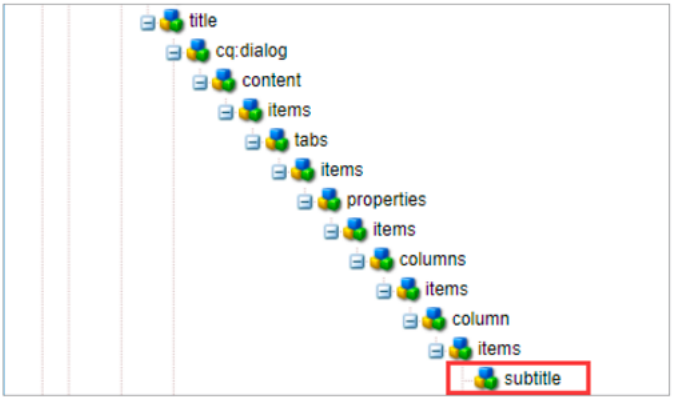
In this exercise, you will extend a core component without losing the functionality that it provides and ensuring it works for future core component upgrades.
To extend the Title component with the Sling Resource Merger:

1. In CRXDE Lite, navigate to the **/apps/training/components/content** folder. Notice that a title proxy component is available.

2. Navigate to the **/apps/core/wcm/components/title/v2/title** node, select the **cq:dialog** node, and click Copy from the actions bar.
3. Navigate to the **/apps/training/components/content/title** node, and click Paste. The **cq:dialog** node is added to the title proxy component.
4. Click Save All from the actions bar.
5. Navigate to the **/apps/training/components/content/title/cq:dialog/content/items/tabs/items/properties/items/columns/items/column/items** node, select the types node, and click Delete from the actions bar.
6. Click Save All from the actions bar.
7. Similarly, select the **defaulttypes** node that is below the items node, and click Delete from the actions bar.
8. Click **Save All** from the actions bar.

Note: You must delete the form fields that you want to inherit from the v2 title component. As a result, the custom formfields you create next will be merged with the v2 title dialog.

9. Select the **title** node, click Rename from the actions bar, and rename it to **subtitle**, as shown:



10. Update the following properties of the **subtitle** node:

Name	Type	Value
fieldLabel	String	Subtitle
fieldDescription	String	Enter your custom subtitle
name	String	./subtitle

11. Click Save All.
12. Open the English page of **We.Train** site and click the Toggle Side Panel icon from the page toolbar. The panel opens.
13. Click the **Components** icon. The **Component** browser opens.

14. Drag the **Title** component from the browser below the **Stockplex** component. The Title component is added to the page.
15. Select the **Title** component a and click the **Configure** (wrench) icon from the component toolbar. The **Title** dialog box opens.
16. Notice that the **Subtitle** field is now added to the dialog box.
17. Update the following details:

    a. Title: My new title

    b. Type / Size: H1

    c. Subtitle: My custom subtitle
18. Click the **Done** (checkmark) icon The English page opens.

Notice how at this point, you do not see the subtitle on the page. You must create a local title.html script to reference the new subtitle property.

19. In **CRXDE Lite**, navigate to the **/apps/training/components/content** folder, select the title node, and click **Create** > **Create** File. The **Create File** dialog box opens.
20. Enter **title.html** in the **Name** field, and click OK. The file is created below the title node.
21. Click **Save All**.
22. Double-click the title.html. The editor opens on the right.
23. Copy the code from **title.html** exercise file provided to you, and paste it in title.html.
24. Click Save All from the actions bar.
25. Open the title.html to view the code that you added. Notice that the script is the exact same script as the core title component, except there is a new line of code that adds the subtitle to the component:

```
1.  <h1 data-sly-use.title="com.adobe.cq.wcm.core.components.models.Title"
2.      data-sly-use.template="core/wcm/components/commons/v1/templates.html"
3.      data-sly-element="${title.type}"
4.      data-sly-test.text="${title.text}">${title.text}</h1>
5.
6.      <!--/* Subtitle added to HTL */-->
7.      <h2 data-sly-test="${properties.subtitle}">${properties.subtitle}</h2>
8.
9.  <sly data-sly-call="${template.placeholder @ isEmpty=!text}"></sly>
```

26. Open the **English** page and observe that the new subtitle is now appearing. You have successfully extended the component. You added a new feature, a subtitle, to the core Title component by using the Sling resource merger to modify its dialog, and a minimal script override to modify its output.

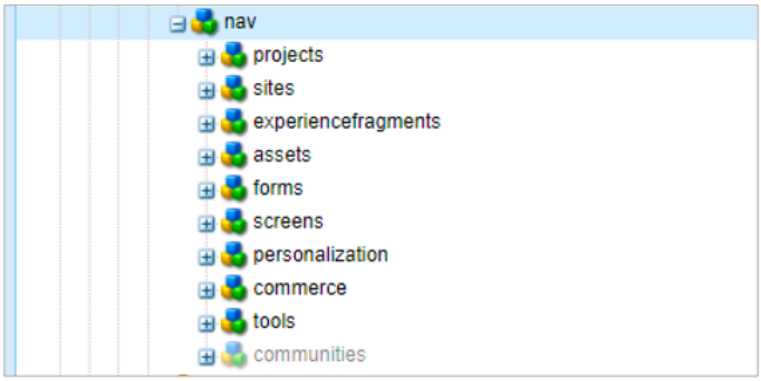
Exercise 2: Extend the navigation UI

In this exercise, you will modify the navigation UI buttons by using the Sling Resource Merger.

1. Click **http://localhost:4502/aem/start.html**. The Navigation UI opens. Notice the Sites navigation button.

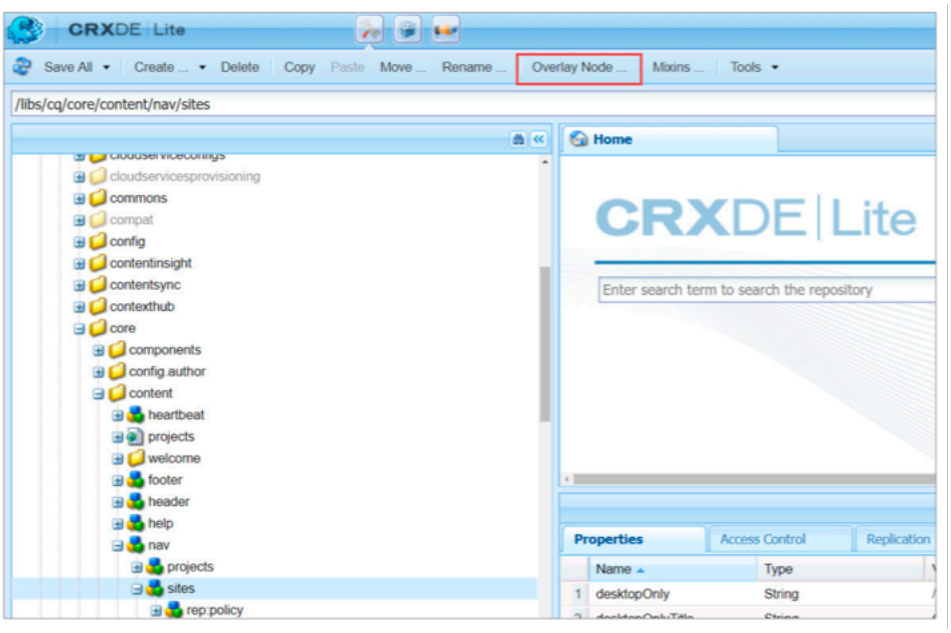
To modify the navigation button:

2. In CRXDE Lite navigate to the **/libs/cq/core/content/nav** node. Notice, how the child nodes of nav are the definition of the global navigation icons (consoles) for AEM.



To modify the navigation icons, hide icons, or add a new icon, you must change the list under **/libs/cq/core/content/nav**. It is a best practice to recreate the structure in /apps by using the Sling Resource Merger to avoid any changes to the **/libs** structure.

3. Navigate to the /libs/cq/core/content/nav node, select the sites node, and click Overlay Node from the actions bar, as shown. The Overlay Node dialog box opens.



4. Ensure the **Overlay Location** field has **/apps/** as its value, select the **Match Node Types** checkbox, and click **OK**.

5. Click **Save All** from the actions bar.

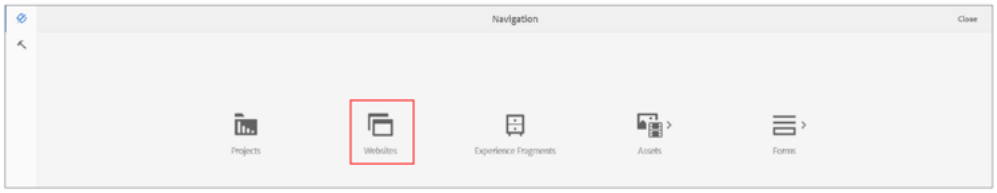
6. Navigate to the **/apps** folder, and notice that the **/cq/core/content/nav/sites** structure is created. If you cannot view the **/apps/cq** folder structure, refresh the **CRXDE Lite** page.

Note: The overlay node structure is reproduced, but the properties are not created. Remember, the Sling Resource Merger will merge properties and nodes.

7. Add the following property to sites under the /apps /cq/core/content/nav/ node:

Name	Type	Value
jcr:title	String	Websites

8. Click **Save All** from the actions bar.
9. Open the tab where the **Navigation** page is open. Refresh the page and notice that the **Sites** button is now renamed to **Websites,** as shown. You used the Sling Resource merger to modify the AEM application navigation.



Exercise 3 : Hide a navigation button

To hide a navigation button:

1. In **CRXDE Lite**, navigate to the **/apps/cq/core/content/nav** node, click Create > Create Node. The **Create Node** dialog box opens.
2. Type **communities** in the **Name** field, and ensure the Type is **nt:unstructured**.
3. Click **OK**. The node is created. Click Save All from the actions bar.
4. Add the following property to the communities node:

Name	Type	Value
sling:hideResource	Boolean	true

5. Click **Save All** from the actions bar.
6. Open the tab where the **Navigation** page is open. Refresh the page and notice that the **Communities** option is hidden from **Navigation**.

Exercise 4: Add a new navigation button

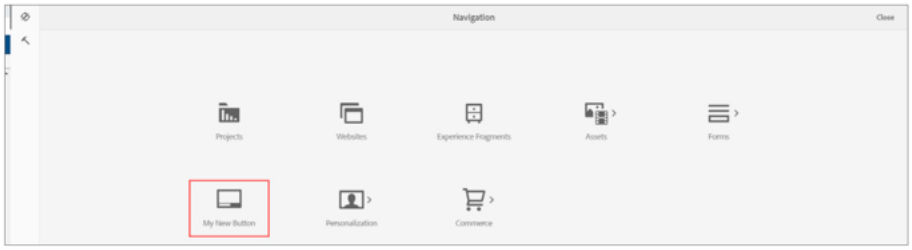
To add a new navigation button:

1. In CRXDE Lite, navigate to the /libs/cq/core/content/nav node, select the screens node, and click Copy from the actions bar.

2. Navigate to the /apps/cq/core/content/nav node, and click Paste from the actions bar. The screens node is added.
3. Click Save All from the actions bar.
4. Select the screens node, and rename it as mybutton.
5. Click Save All from the actions bar.
6. Update the following properties of the mybutton node:
7.

Name	Type	Value
jcr:title	String	My New Button
id	String	cq-mybutton
href	String	/sites.html

8. Click Save All from the actions bar. Open the tab where the Navigation page is open. Refresh the page and notice that a new console, My New Button, is created, as shown:



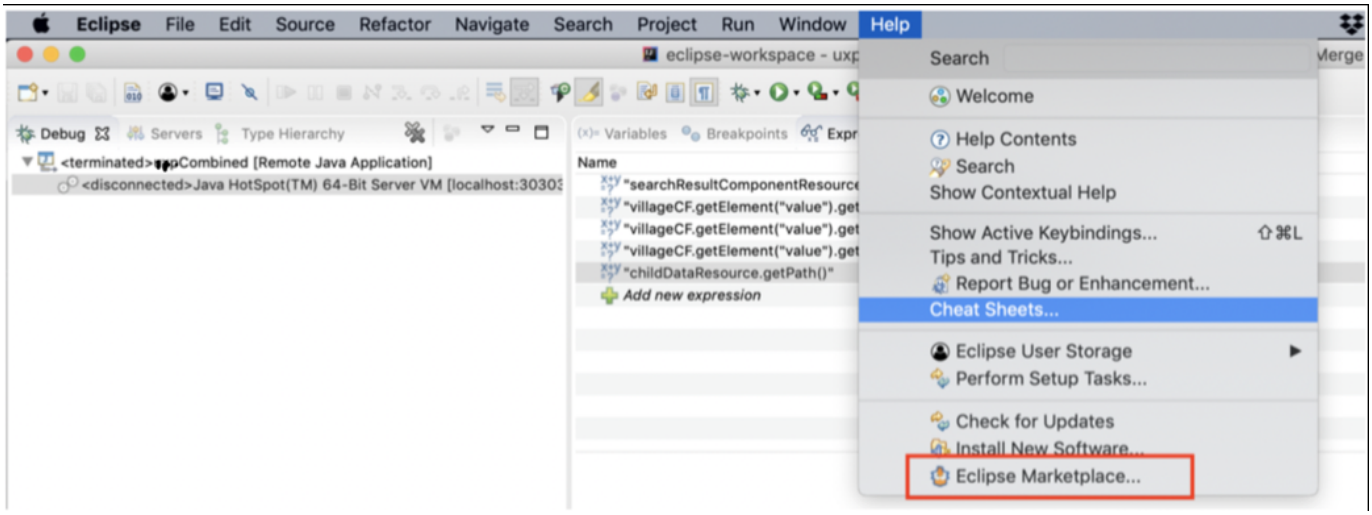
9. Click My New Button. Notice that the Sites console opens.

Eclipse Installation

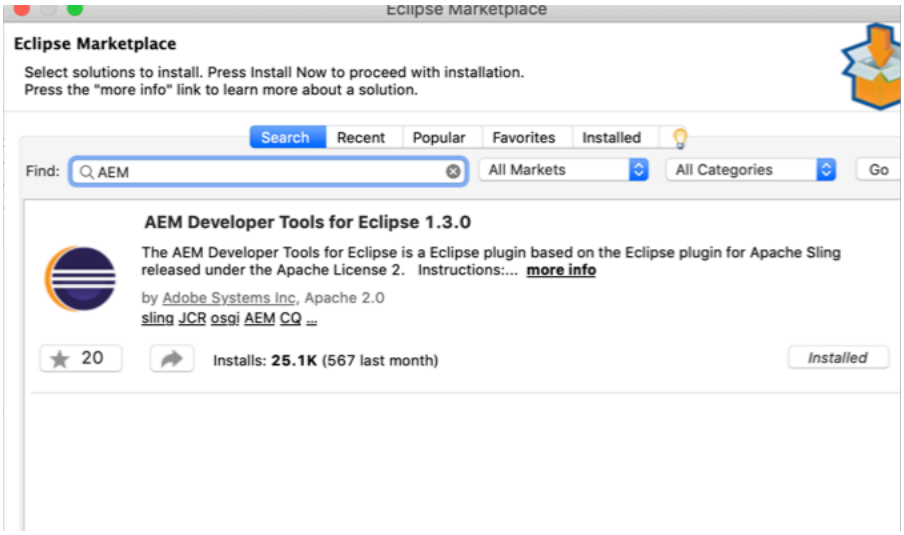
For this follow the internet.

AEM plugin Installation into Eclipse

Go to eclipse marketplace and install AEM plugin as shown:

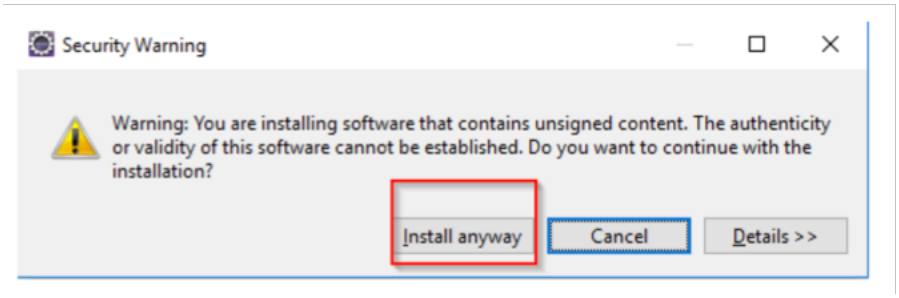


After opening Eclipse Marketplace, you will see a popup. In this popup, enter AEM in the Find bar. It will search through the web and will display the AEM plugin for eclipse. Click on **Install**.



Accept the License agreement and **Finish**

The eclipse will take some time to install the software and will show a popup. Click on **Install Anyway**.

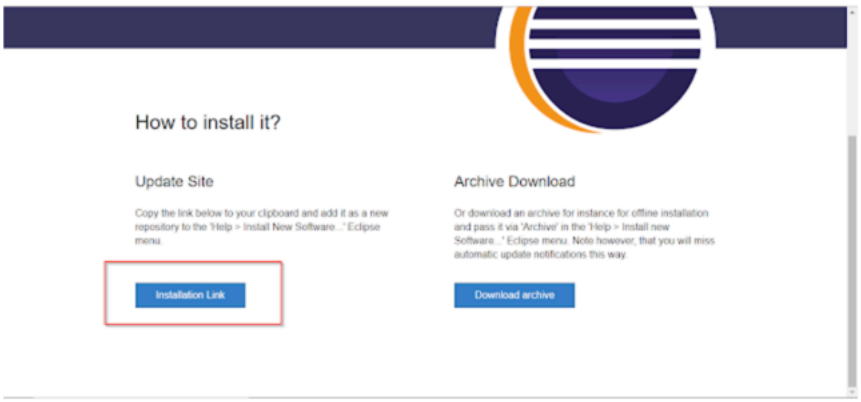


Once the software is installed, Restart the Eclipse to reflect changes.

Installing AEM Developer Tools

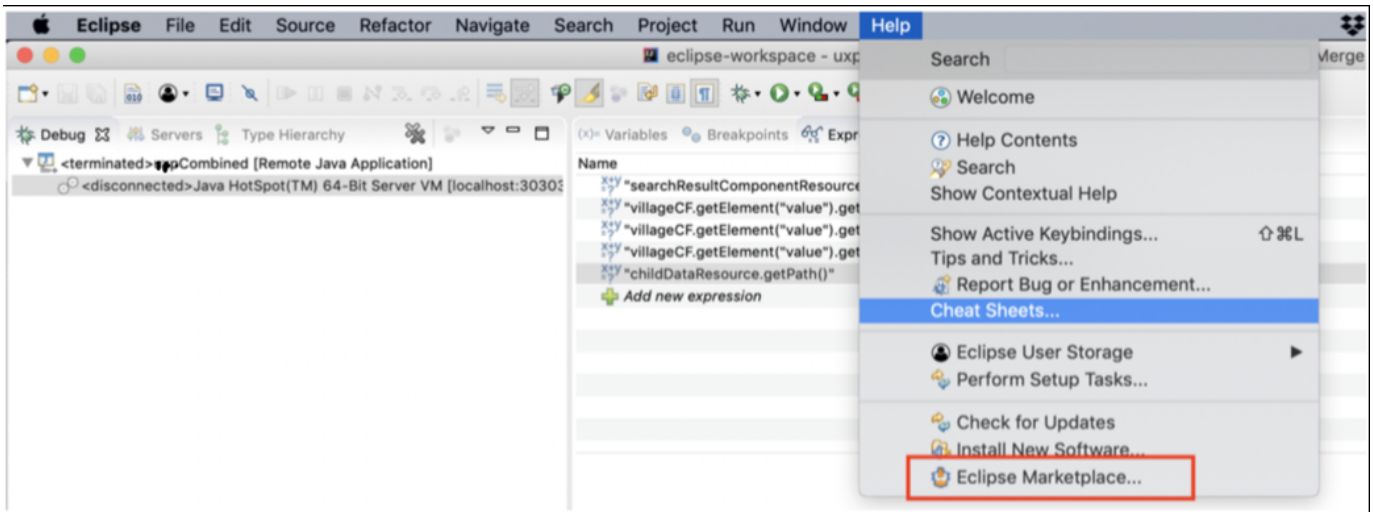
AEM developer tools provide us with many features such as integration of AEM server and eclipse, sync of content and bundles, debugging, modification of JCR properties and many more.
To install AEM Developer tools, follow the below steps -

1. Go to AEM Developer Tools site.
2. You will see an **Installation Link.** Copy it.



Installation Link

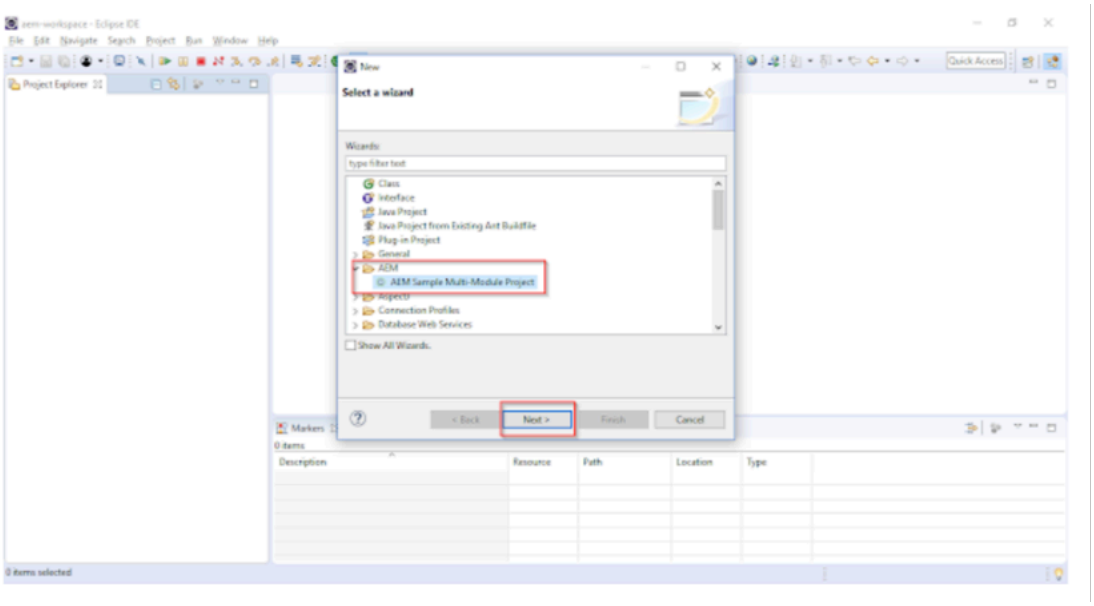
3. In Eclipse, go to Help, then Install New Software...



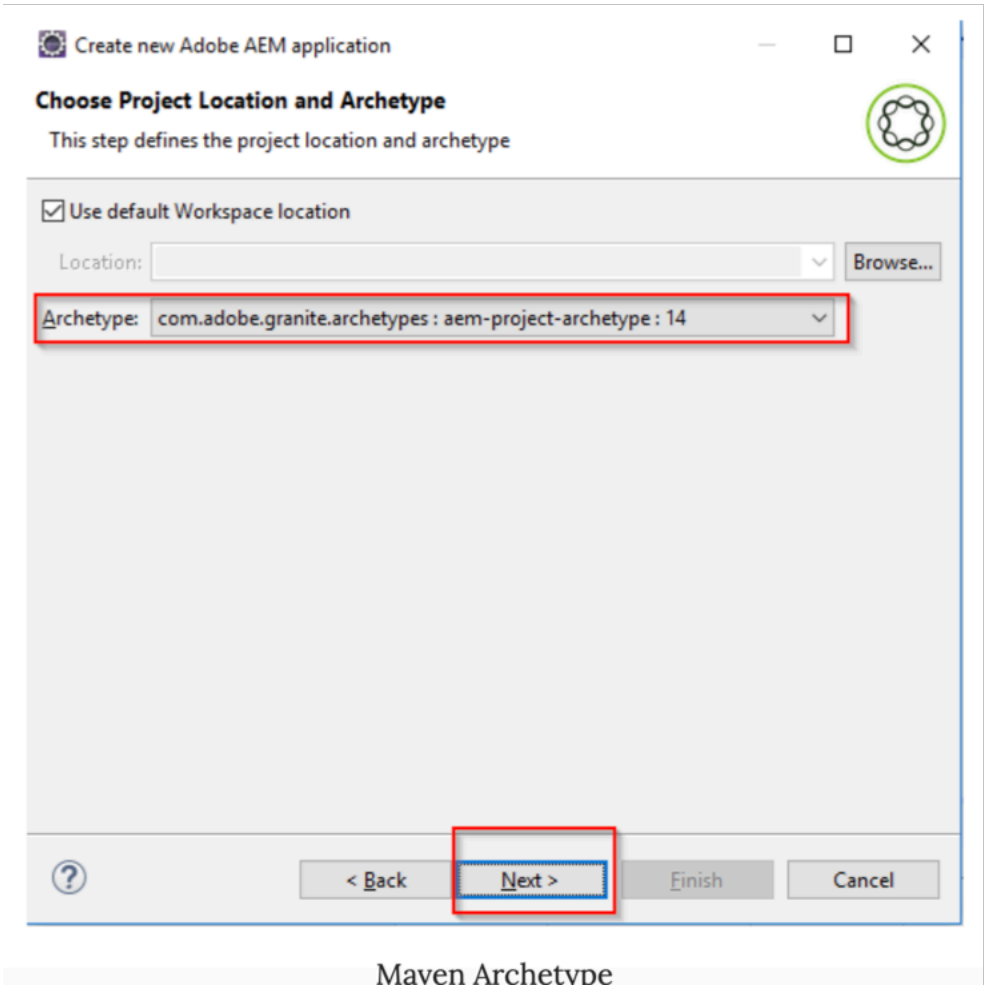
4. Click Add..., then in the name field, put "AEM Developer Tools" and in the location field. paste the installation link.
5. Click Next, Next and Finish. Restart the eclipse to so the new changes are reflected.

Exercise 5: Create an AEM project using latest archetype in eclipse.

1. Go to New, then New Project, then Other and select AEM Multimodule Project and click Next

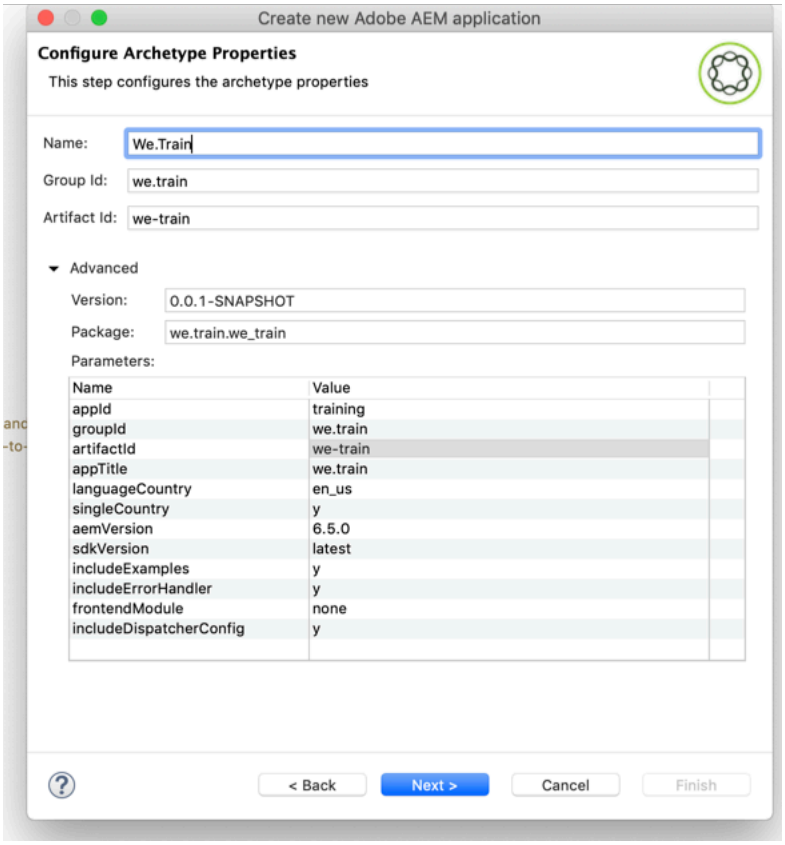


2. Select latest Archetype (which is 23 as of now), click next.

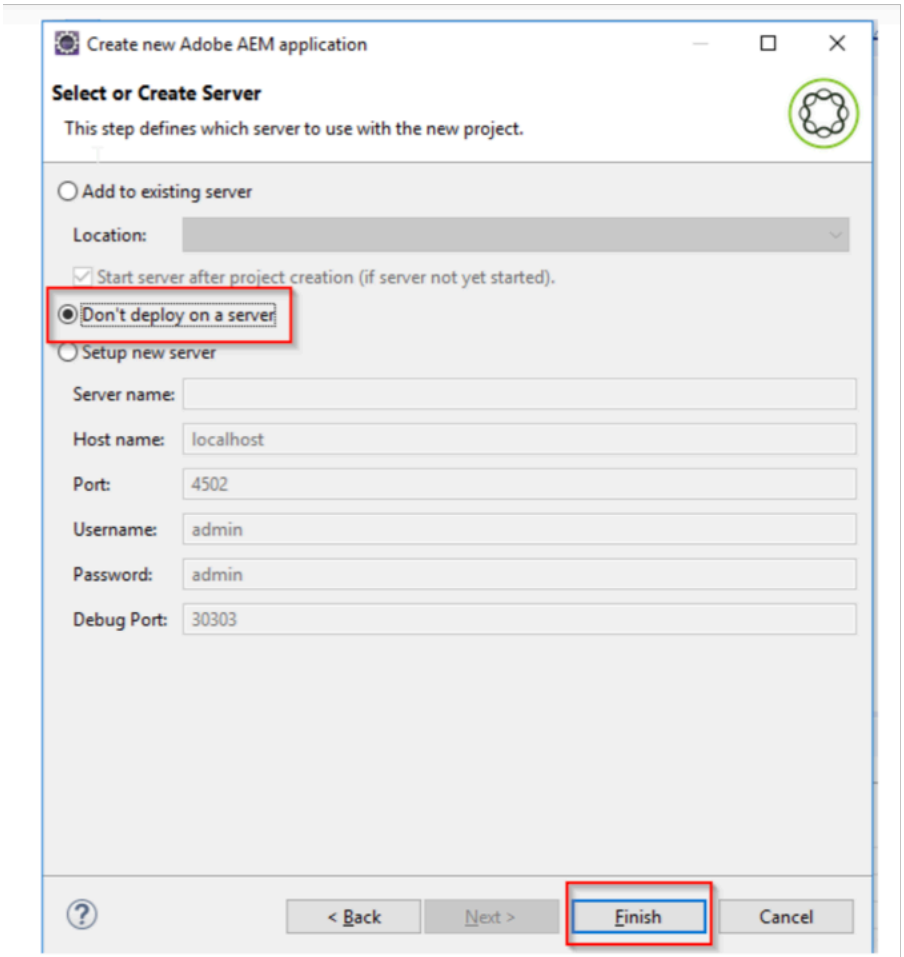


Maven Archetype

3. Put the required fields as shown in the figure and click on Next



4. Select Don't deploy on a server and click Finish



Finish

The project will be created will default template. This will have the following children projects - core, launcher, tests, apps and content.